

1. In this quiz we will consider Lagrange multipliers as a technique to find a minimum of a function subject to a constraint, i.e. solutions lying on a particular curve.

1 point

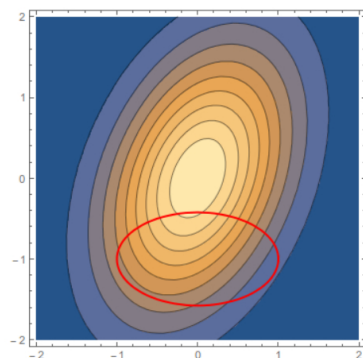
Let's consider the example of finding the minimum of the function,

$$f(\mathbf{x}) = \exp\left(-\frac{2x^2 + y^2 - xy}{2}\right)$$

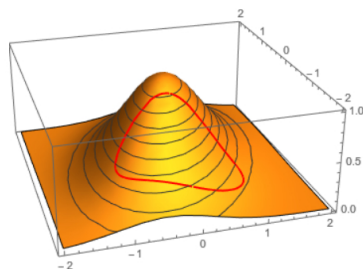
along the curve (or, subject to the constraint),

$$g(\mathbf{x}) = x^2 + 3(y+1)^2 - 1 = 0.$$

The functions themselves are fairly simple, on a contour map they look as follows,



However, their combination can become quite complicated if they were computed directly, as can be inferred from the shape of the constraint on the surface plot,



Do note, in this case, the function $f(\mathbf{x})$ does not have any minima itself, but along the curve, there are two minima (and two maxima).

A situation like this is where Lagrange multipliers come in. The observation is that the maxima and minima on the curve, will be found where the constraint is parallel to the contours of the function.

Since the gradient is perpendicular to the contours, the gradient of the function and the gradient of the constraint will also be parallel, that is,

$$\nabla f(\mathbf{x}) = \lambda \nabla g(\mathbf{x})$$

If we write this out in component form, this becomes,

$$\begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \lambda \begin{bmatrix} \frac{\partial g}{\partial x} \\ \frac{\partial g}{\partial y} \end{bmatrix}$$

This equation, along with $g(\mathbf{x}) = 0$ is enough to specify the system fully. We can put all this information into a single vector equation,

$$\nabla \mathcal{L}(x, y, \lambda) = \begin{bmatrix} \frac{\partial f}{\partial x} - \lambda \frac{\partial g}{\partial x} \\ \frac{\partial f}{\partial y} - \lambda \frac{\partial g}{\partial y} \\ -g(\mathbf{x}) \end{bmatrix} = 0$$

and then solve this equation to solve the system.

Let's reflect on what we have done here, we have converted from a question of finding a minimum of a 2D function constrained to a 1D curve, to finding the zeros of a 3D vector equation.

Whereas this may sound like we have made the problem more complicated, we are exactly equipped to deal with this kind of problem; we can use the root finding methods, such as the Newton-Raphson method that we've discussed previously.

Let's set up the system,

The function and two of the derivatives are defined for you. Set up the other two by replacing the question marks in the following code.

```
1 # First we define the functions,
2 def f(x, y):
3     return np.exp(-(2*x*x + y*y - x*y) / 2)
```

```

4
5 def g(x, y):
6     return x*x + 3*(y+1)**2 - 1
7
8 # Next their derivatives,
9 def dfdx(x, y):
10    return 1/2 * (-4*x + y) * f(x, y)
11
12 def dfdy(x, y):
13    return 1/2 * (x - 2*y) * f(x, y)
14
15 def dgdx(x, y):
16    return 2 * x
17
18 def dgdy(x, y):
19    return 6 * (y+1)

```

2. Next let's define the vector, $\nabla \mathcal{L}$, that we are to find the zeros of; we'll call this "DL" in the code. Then we can use a pre-written root finding method in scipy to solve.

1 point

```

1 from scipy import optimize
2
3 def DL(xyλ):
4     [x, y, λ] = xyλ
5     return np.array([
6         dfdx(x, y) - λ * dgdx(x, y),
7         dfdy(x, y) - λ * dgdy(x, y),
8         -g(x, y)
9     ])
10
11 (x0, y0, λ0) = (-1, -1, 0)
12 x, y, λ = optimize.root(DL, [x0, y0, λ0]).x
13 print("x = %g" % x)
14 print("y = %g" % y)
15 print("λ = %g" % λ)
16 print("f(x, y) = %g" % f(x, y))

```

Run

Reset

Here, the first two elements of the array are the x and y coordinates that we wanted to find, and the last element is the Lagrange multiplier, which we can throw away now it has been used.

Check that (x, y) does indeed solve the equation $g(x, y) = 0$.

You should be able to use the code find the other roots of the system.

Re-use the code above with different starting values to find the other stationary points on the constraint.

There are four in total. Give the y coordinate of any of the other solutions to two decimal places.

-0.43

3. In the previous question, you gave the y coordinate of any of the stationary points. In this part, give the x coordinate of the global minimum of $f(x)$ on $g(x) = 0$.

1 point

Give your answer to 2 decimal places.

0.93

4. You may be wondering about why the vector $\nabla \mathcal{L}$ gets a funny symbol. This is because it can be written as the gradient (over x, y , and λ) of a scalar function $\mathcal{L}(x, y, \lambda)$.

1 point

Based on your knowledge of derivatives, what function $\mathcal{L}(x, y, \lambda)$ would give the expected form of $\nabla \mathcal{L}$?

- ☒ $\mathcal{L}(x, y, \lambda) = f(x) - \lambda g(x)$
- ☐ $\mathcal{L}(x, y, \lambda) = f(x) + g(x) + \lambda$
- ☐ $\mathcal{L}(x, y, \lambda) = f(x) + \lambda g(x) + \lambda$
- ☐ $\mathcal{L}(x, y, \lambda) = f(x, y) + g(x, y, \lambda)$

5. Hopefully you've now built up a feeling for how Lagrange multipliers work. Let's test this out on a new function and constraint.

1 point

Calculate the minimum of

$$f(x, y) = -\exp(x - y^2 + xy)$$

on the constraint,

$$g(x, y) = \cosh(y) + x - 2 = 0$$

Use the code you've written in the previous questions to help you.

```

1 # Import libraries
2 import numpy as np
3 from scipy import optimize
4

```

```

5 # First we define the functions, YOU SHOULD IMPLEMENT THESE
6 def f (x, y) :
7     return -np.exp(x - y*y + x*y)
8
9 def g (x, y) :
10    return np.cosh(y)+x-2
11
12 # Next their derivatives, YOU SHOULD IMPLEMENT THESE
13 def dfdx (x, y) :
14    return (-np.exp(x - y*y + x*y))*(1+y)
15
16 def dfdy (x, y) :
17    return (-np.exp(x - y*y + x*y))*(-2*y+x)
18
19 def dgdx (x, y) :
20    return 1
21
22 def dgdy (x, y) :
23    return np.sinh(y)
24
25 # Use the definition of DL from previously.
26 def DL (xyλ) :
27     [x, y, λ] = xyλ
28     return np.array([
29         dfdx(x, y) - λ * dgdx(x, y),
30         dfdy(x, y) - λ * dgdy(x, y),
31         - g(x, y)
32     ])
33
34 # To score on this question, the code above should set
35 # the variables x, y, λ, to the values which solve the
36 # Langrange multiplier problem.
37
38 # I.e. use the optimize.root method, as you did previously.
39 (x0, y0, λ0) = (0, 0, 0)
40 x, y, λ = optimize.root(DL, [x0, y0, λ0]).x

```